

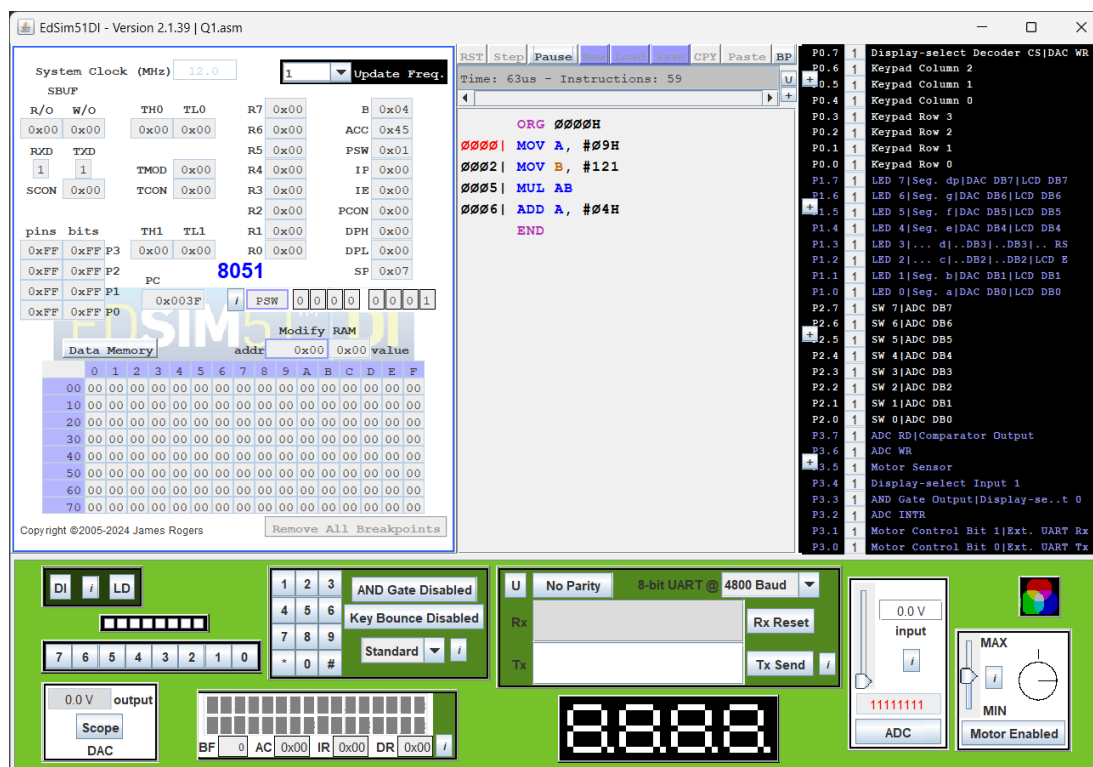
MES ASSIGNMENT CA-1

NAME: Harsh Kumar

PRN: 24070521093

SEM: IV, B, 2024-28

Q1. Write an 8051 Assembly Language Program (ALP) to generate the last four digits of your PRN using any arithmetic instructions. The program should not directly load the complete PRN number as an immediate value. Instead, it must use appropriate arithmetic operations such as ADD, MUL, or INC to form the number logically. The final result must be stored in the Accumulator register (AX). For example, if a student's PRN is 24070521091, the last four digits are 1091, and the value 1091 should be available in AX at the end of program execution.



The program constructs the number 1093 step by step using arithmetic operations instead of loading it directly. Initially, two small values are added in the accumulator to produce the number 9. This value is then multiplied by 126 using the MUL AB instruction. Since the multiplication results in a 16-bit output, the lower byte is stored in the Accumulator while the higher byte is stored in register B. After that, an additional value of 5 is added to the accumulator to obtain the final result 1093. Hence, the required last four digits are generated logically and the complete result is present in the AX (A and B registers).

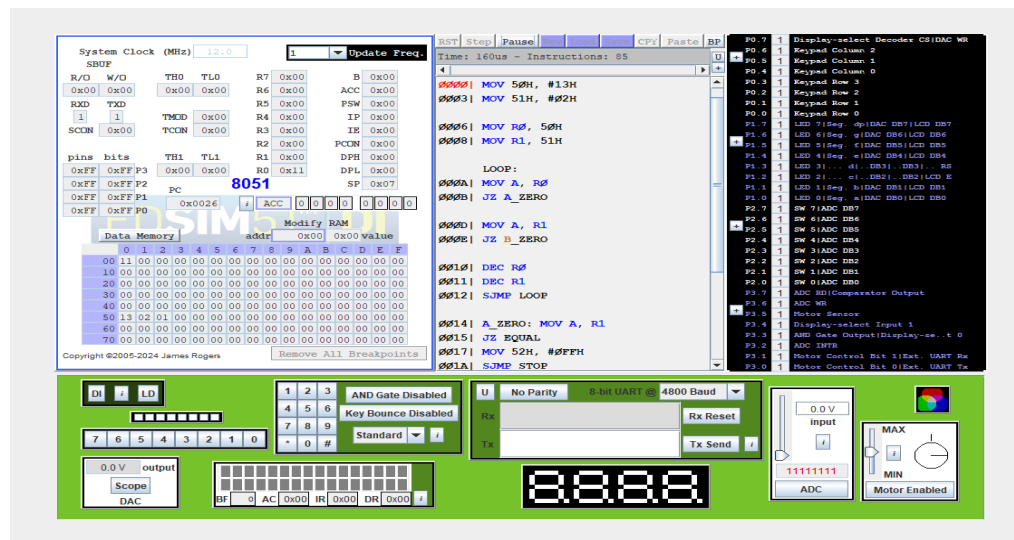
NAME: Harsh Kumar

PRN: 24070521093

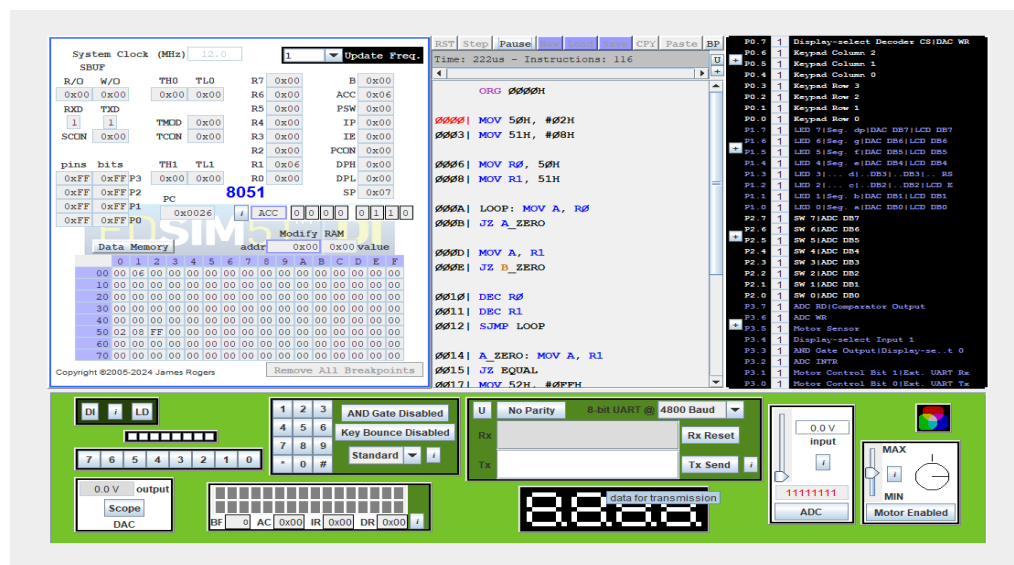
SEM: IV, B, 2024-28

Q.2 Execute an 8051-assembly language program for a safety-certified system in which the instructions CJNE, DJNZ, and SUBB are not permitted. Two unsigned numbers are stored in internal RAM locations 50H and 51H. The program must compare these two numbers using only the allowed instruction set (MOV, INC, DEC, JZ, JNZ, CLR, SETB, ANL, ORL) and store the comparison result in a register or memory location such that 01H indicates the value at 50H is greater than the value at 51H, 00H indicates both values are equal, and FFH indicates the value at 50H is less than the value at 51H. The program should be simulated for all three possible cases ($A > B$, $A = B$, $A < B$), and the solution must clearly explain how flag behavior (especially the Zero flag) is utilized to achieve comparison under the given instruction constraints.

For $A > B$



For $A < B$



SEM: IV, B, 2024-28

The screenshot displays the Proteus 8.0 SP3 software interface for a PIC16F887 microcontroller project. The top window shows the assembly code, and the bottom window shows the hardware configuration.

Assembly Code (PIC16F887.asm):

```

System Clock (MHz) 12.0
Update Freq. 1
RST Step Pause New Load Save CPU Paste BP
Time: 110us - Instructions: 66

ORG 0000H
MOV 50H, #05H
MOV 51H, #05H
MOV R0, 50H
MOV R1, 51H
LOOP: MOV A, R0
JZ A_ZERO
MOV A, R1
JZ B_ZERO
DEC R0
DEC R1
SUMP LOOP
A_ZERO: MOV A, R1
JZ EQUAL
MOV 52H, #0FFH

```

Hardware Configuration:

- PIC16F887:** Microcontroller chip.
- 10k Potentiometer:** Connected to the PIC16F887.
- 7-Segment Display:** Connected to the PIC16F887.
- 0.0V Output:** Scope DAC output.
- ADC:** Analog-to-Digital Converter.
- Motor:** Motor Enabled.

The status bar at the bottom indicates the project is running.

In this program, two unsigned numbers stored at memory locations 50H and 51H are compared without using the instructions CJNE, DJNZ, and SUBB. The values are first moved into registers R0 and R1. Both registers are then decreased together in a loop. After each step, the Zero flag is checked using the JZ instruction. If R1 becomes zero first while R0 is still not zero, the program stores 01H in memory location 52H, which means $A > B$. If R0 becomes zero first while R1 is still not zero, the program stores FFH in location 52H, which means $A < B$. If both R0 and R1 become zero at the same time, the program stores 00H in location 52H, which means $A = B$. In this way, the comparison is done using only the allowed instructions and by using the Zero flag.

NAME: Harsh Kumar

PRN: 24070521093

SEM: IV, B, 2024-28

Q.3 A student claims that two assembly programs are equivalent because both access the same RAM address; however, this claim is incorrect due to the difference in addressing modes. In this case study, write two short assembly programs—one using direct addressing and the other using indirect addressing—such that both reference the same RAM location. Using an appropriate initial RAM configuration, demonstrate a situation where the outputs of the two programs differ even though the base address is the same. Support the observation with register and RAM snapshots from simulation, and explain that the difference arises because direct addressing accesses the data stored at the given address, whereas indirect addressing treats the contents of that address as a pointer to another memory location, leading to different data being fetched and hence different outputs.

The screenshot displays the Proteus simulation environment. The central window shows an assembly program with the following code:

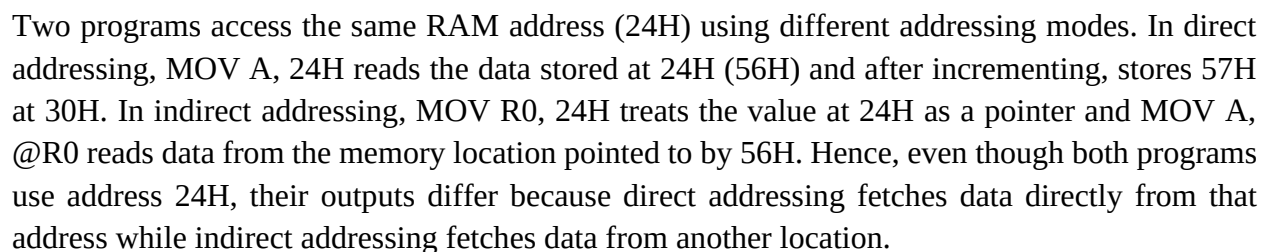
```
ORG 0000H
0000| MOV 24H, #56H
0003| MOV A, 24H
0005| INC A
0006| MOV 30H, A
END
```

The left panel shows the I/O View with various registers and pins. The PC register is highlighted at 8051. The Data Memory window shows a table of memory addresses and values:

addr	0x00	0x01	...	0xFF
00	00	00	...	00
10	00	00	...	00
20	00	00	...	00
30	57	00	...	00
40	00	00	...	00
50	05	05	...	00
60	00	00	...	00
70	00	00	...	00

The right panel shows the I/O View with various components and their states. The bottom panel shows the hardware components, including a DAC, a scope, a UART, and a motor.

SEM: IV, B 2024-28



NAME: Harsh Kumar

PRN: 24070521093

SEM: IV, B 2024-28

Q.4 Write an 8051 Assembly Language Program in which you must use logical instructions to construct a numeric result. Using multiple logical instructions such as ANL, ORL, and CLR, generate the last four digits of your own mobile number through a suitable sequence of operations (you may split the digits and combine them logically as required). Do not directly load the complete 4-digit number as an immediate value. The program should use more than one logical instruction, and at the end of execution the Accumulator (A) must contain the last four digits of your mobile number. Simulate the program and verify that the final value in the Accumulator matches your mobile number's last four digits.

The screenshot displays the EdSim51DI simulation environment. The main window shows the assembly code being executed:

```
ORG 0000H
0000| MOV A, #FFH
0002| ANL A, #1EH
0004| MOV R0, A
0005| MOV A, #FFH
0007| ANL A, #0D6H
0009| MOV R1, A
000A| MOV B, R0
000C| MOV A, R1
END
```

The register window shows the state of the 8051 microcontroller:

Register	Value
R0	0x1E
R1	0xD6
R2	0x00
R3	0x00
R4	0x00
R5	0x00
R6	0x00
R7	0x00
ACC	0xD6
PSW	0x01
IP	0x00
IE	0x00
PCON	0x00
DPH	0x00
DPL	0x00
SP	0x07

The code memory window shows the program being loaded into memory starting at address 0x0000. The right-hand pane displays the hardware configuration, including the display, keypad, and various peripheral modules. The bottom status bar shows the current state of the simulation, including the system clock (12.0 MHz) and the execution time (16us).

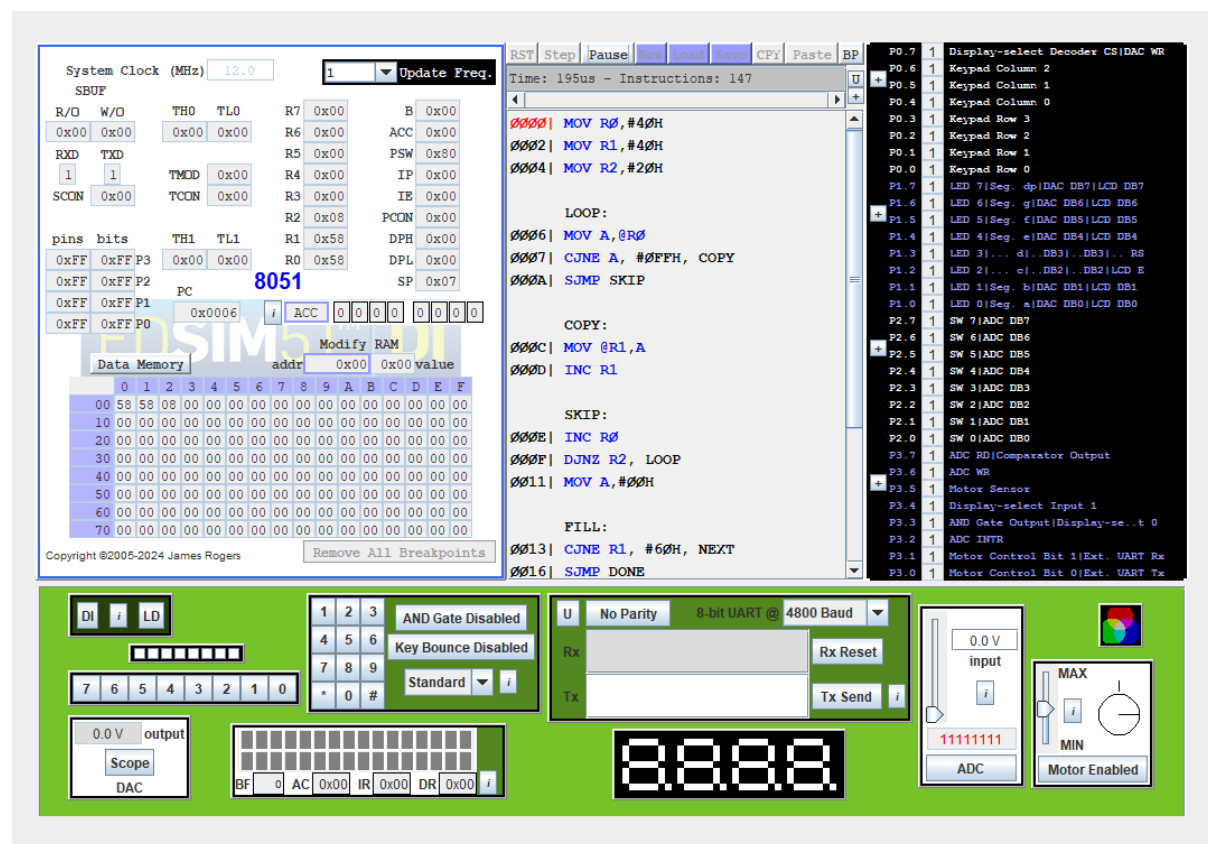
In this program, the last four digits of the mobile number (7894) are generated using logical instructions instead of directly loading the value. The number is divided into a high byte and a low byte. First, the accumulator is loaded with FFH and masked using the ANL instruction to form the high byte (1EH). This value is stored in register R0. Then, the accumulator is again loaded with FFH and masked using ANL to form the low byte (D6H), which is stored in register R1. Finally, the high byte is moved to register B and the low byte is moved to the accumulator. Thus, the combined value in the AX (B:A) register pair represents 7894, verifying that the required last four digits are generated using logical instructions.

NAME: Sparsh Goswami

PRN: 24070521091

SEM: 4th, B (2024-28)

Q.5 An embedded logger stores event codes in internal RAM from 40H to 5FH, but due to strict memory limitations the data must be compacted in-place without using any additional RAM or the stack. Write an assembly language program that scans the memory range 40H–5FH using only indirect addressing, removes all occurrences of the value FFH, shifts the remaining valid data bytes to the left to eliminate gaps, and fills the unused memory locations at the end of the range with 00H. **Execute** the program to show the RAM contents before and after execution, and clearly explain the pointer movement logic used to identify valid data, shift it correctly, and overwrite invalid entries under the given constraints.



This program scans the internal RAM from address 40H to 5FH using indirect addressing. The read pointer (R0) checks each memory location and compares the data with FFH. If the value is not FFH, it is copied to the location pointed by the write pointer (R1), thereby shifting valid data to the left and removing gaps. After all bytes are processed, the remaining memory locations are filled with 00H. Thus, all FFH values are removed, valid data is compacted, and unused locations are cleared.

Github repo link - <https://github.com/Harsh2227kumar/MES-CA-I/>